

Илья Волков. Построение распределённых систем: подходы, методы, команда

Senior Software Engineer, ex-Binance, ex-BP. Java, Kotlin, Scala.

Цель

- System Design - зачем нужен, верхнеуровневый план построения;
- Разобрать процесс на примере - создание приложения, похожего на Tinder

Теория

Почему важно уметь проектировать системы?

- Проектирование новых систем, подсистем, сервисов
- Понимание современных подходов к построению распределенных систем
- Интервью

Системный дизайн на интервью и в жизни

- Интервью: ограничено по времени, фокус на структуре и идеях
- Реальные проекты: ограничения, стоимость, время, поддержка, эволюция

План

1. Понять задачу (функциональные требования)
2. Определить нефункциональные требования
3. Приблизительные оценки нагрузки
4. Модель данных и ключевые сущности
5. Архитектура высокого уровня
6. Детализация ключевых компонентов
7. Масштабирование и отказоустойчивость

(сбор статистики, отчетов, мониторинг, безопасность системы)

Практика

Пример: приложение для знакомств

- Проектируем систему, похожую на Tinder
- Задача - соединять пользователей на основе профилей с использованием AI

Функциональные требования

- Регистрация и редактирование профиля
- Лайки и дизлайки
- Поиск и рекомендации

Не рассматриваем: чат пользователей, защиту от ботов, поддержку видео

Нефункциональные требования

- 20 млн. зарегистрированных пользователей (10 млн. активных)
- Получение стека свайпов <300 мс
- Доступность (99.9%)
- Не показываем пользователю отклоненные матчи (consistency)

Оценки нагрузки

- 10 млн. активных пользователей в день (10^7)
- ~100 свайпов в день -> 1 млрд. свайпов в сутки (10^9)
- ~11500 свайпов в секунду (10^4)
- Чтение/запись ~80/20

Модель данных

- User(id, name, age, gender, location)
- Profile(userId, interests, photos)
- Swipe(userId, targetId, direction, timestamp)
- Match(user1, user2, timestamp)



Основные компоненты системы



Высокоуровневая архитектура



Высокоуровневая архитектура 2



Высокоуровневая архитектура 3



Высокоуровневая архитектура 4

Сервис подбора (Matching Service)

- Определяет потенциальных кандидатов для пользователя
- Фильтры: возраст, местоположение, интересы

- ML-модель ранжирует кандидатов по вероятности совпадения
- Система должна работать в реальном времени

Алгоритм подбора

1. Почить кандидатов из базы (по локации и полу)
2. Отфильтровать по предыдущим свайпам
3. Передать в ML модель для ранжирования
4. Вернуть топ-N кандидатов

Использование ML

- Модель обучается на исторических данных о свайпах
- Фиши: интересы, расстояние, активность, время суток
- Модель оценивает вероятность лайка и формирует рекомендации

Хранение данных

- Профили - SQL (PostgreSQL)
- Свайпы - NoSQL
- Фото - Object Storage (S3)
- Кэширование - Redis

Масштабирование

- Горизонтальное масштабирование сервисов
- Шардирование по регионам
- Балансировка нагрузки
- Кэширование популярных пользователей
- Асинхронная обработка (Kafka)

Отказоустойчивость

- Репликация данных
- Health-check и auto-restart контейнеров
- Circuit Breaker для внешних зависимостей
- Monitoring (Prometheus, Grafana)

Безопасность

- Аутентификация и авторизация (OAuth 2.0)
- Шифрование данных (TLS, AES)
- Защита персональных данных

Мониторинг и метрики

- Ошибки, задержка, загрузка CPU/памяти
- Количество активных пользователей
- Метрики ML-модели
- Логи и алерты

Типичные компромиссы

- Consistency vs Availability
- Latency vs Accuracy
- Простота реализации vs Гибкость
- Стоимость vs Производительность

Что важно помнить?

Начинайте с требований, а не с технологий. Простое решение -> масштабирование.
Trade-offs. Документация архитектурных решений.

Инструменты и технологии

- Языки: Java, Kotlin, Go
- Базы: PostgreSQL, Cassandra, Redis
- Очереди: Apache Kafka
- Облако: Amazon Cloud, Azure
- Контейнеризация: Docker, Kubernetes

Как готовиться к System Design интервью?

- Практика (проектирование существующих систем)
- Разбор конкретных технологий в деталях (pet-проекты)
- Практикуемся в написании компонент самостоятельно (`codecrafters.io`)

1. <https://github.com/donnemartin/system-design-primer>
2. <https://www.hellointerview.com/learn/system-design/in-a-hurry/introduction>
3. Книжка с кобанчиком (Мартин Клеппман)